Python for Data Science

A 5-Hour Study Plan

NumPy, Pandas, Matplotlib & Seaborn, File I/O, and the Python habits every data scientist needs

Module 1 of the Data Science & ML Training Series

Paired with: *Module 1 Practice Notebook* and the *Mini Project: EDA on India's COVID-19 Data*

Handbook prepared for postgraduate and BCA Data Science training programs

Contents

How to Use This Handbook

This handbook is built for a single focused **5-hour study session**. Each section below tells you roughly how long to spend on it. Read the idea, run the code, and don't worry about memorising anything — the practice notebook that comes with this handbook lets you actually type and run every example on a real dataset (IPL match data).

#	Topic	Time	What you'll be able to do
1	Getting Ready — Virtual Environments & Jupyter	15 min	Set up a clean, isolated project every time
2	Python Habits: Comprehensions, Lambda, Map/Filter	45 min	Write short, fast, "Pythonic" data code
3	NumPy — Arrays & Vectorised Operations	60 min	Do maths on whole columns without loops
4	Pandas — DataFrames, groupby, merge, pivot	75 min	Explore and reshape real tables of data
5	File I/O — CSV, JSON, Excel	45 min	Read data in and save your work out
6	Matplotlib & Seaborn for EDA	60 min	Turn numbers into charts people understand

Note

Total time: **5 hours (300 minutes)**. Take a 5–10 minute break after Section 3 and after Section 5 — your brain absorbs code better in two-hour-ish stretches, not one long sitting.

1 Getting Ready: Virtual Environments & Jupyter

(15 minutes)

1.1 Why bother with a virtual environment?

Imagine you're working on two projects. One needs `pandas` version 1.5, the other needs version 2.1. If you install packages directly on your system, the two projects will fight over which version is installed. A **virtual environment** is simply a private, isolated folder of Python packages for one project, so different projects never clash.

Key Idea

A virtual environment is a self-contained Python installation just for one project. Nothing you install inside it affects any other project on your computer.

1.2 Creating and activating one

```
# Create a virtual environment named "venv"
python -m venv venv

# Activate it (Windows)
venv\Scripts\activate

# Activate it (macOS / Linux)
source venv/bin/activate

# Your terminal prompt now shows (venv) at the start
```

1.3 Installing packages and keeping track of them

```
pip install numpy pandas matplotlib seaborn jupyter openpyxl

# Save the exact list of installed packages to a file
pip freeze > requirements.txt

# Anyone else (or future you) can rebuild the same environment with:
pip install -r requirements.txt
```

1.4 Launching Jupyter

```
jupyter notebook
# or, for the newer interface
jupyter lab
```

Why this matters

Every dataset you touch in this handbook (and the mini project) is pulled live from a public CSV on the internet. A clean virtual environment with the right packages means the notebook just runs, first try, on any machine.

2 Python Habits You'll Use Constantly

(45 minutes)

Before diving into NumPy and Pandas, it helps to be comfortable with a few compact Python patterns. You will see these everywhere in real data science code, often packed into a single line.

2.1 List comprehensions

A list comprehension is a short way of building a new list from an existing one, instead of writing a full `for` loop.

Key Idea

`[expression for item in iterable if condition]` — read it left to right as "give me *expression*, for every *item* in *iterable*, but only if *condition* is true."

```
marks = [40, 55, 62, 78, 91, 33, 85]

# The long way
passed = []
for m in marks:
    if m >= 40:
        passed.append(m)

# The list-comprehension way -- same result, one line
passed = [m for m in marks if m >= 40]
print(passed)       # [40, 55, 62, 78, 91, 85]

# Transform values while filtering
grades = ["Pass" if m >= 40 else "Fail" for m in marks]
print(grades)
```

2.2 Lambda functions

A `lambda` is a tiny, unnamed function, useful when you need a quick function just once (for example, to tell `pandas` how to sort or transform something).

```
# A normal function
def square(x):
    return x * x

# The same thing as a lambda
square = lambda x: x * x
print(square(5))    # 25

# Most common real use: as a throwaway function passed to another function
students = [("Aisha", 78), ("Ravi", 55), ("Meera", 91)]
students_sorted = sorted(students, key=lambda s: s[1], reverse=True)
print(students_sorted) # sorted by marks, highest first
```

2.3 `map()` and `filter()`

`map()` applies a function to every item; `filter()` keeps only the items that pass a test. Both are close cousins of the list comprehension — pick whichever reads more clearly to you.

```python
marks = [40, 55, 62, 78, 91, 33, 85]

# map(): apply a function to every element
percentages = list(map(lambda m: m / 100, marks))

# filter(): keep only elements where the function returns True
passed = list(filter(lambda m: m >= 40, marks))

print(percentages)
print(passed)
```

Note

In everyday pandas code, list comprehensions are used far more often than `map()`/`filter()` because they read more naturally. Still, you will bump into all three in other people's code, so it's worth recognising them.

3 NumPy — Arrays & Vectorised Operations

(60 minutes)

3.1 Why not just use Python lists?

A plain Python list can hold anything, and that flexibility makes it slow for maths. **NumPy** gives you the `array`: a fixed-type grid of numbers that lets you do maths on the *whole* array at once, without writing a loop.

Key Idea

NumPy arrays let you write `a + b` instead of looping through every element to add them one at a time. This is called **vectorisation**, and it's both shorter to write and much faster to run.

3.2 Creating arrays

```
import numpy as np

marks = np.array([40, 60, 80, 100])
print(marks)          # [ 40 60 80 100]
print(marks.shape)    # (4,)    -- 4 elements, 1 dimension
print(marks.dtype)    # int64

# A 2D array (rows x columns) -- like a small table
scores = np.array([[80, 90], [60, 70], [55, 65]])
print(scores.shape)   # (3, 2) -- 3 students, 2 subjects

# Handy shortcuts
zeros = np.zeros(5)
ones = np.ones((2, 3))
range_arr = np.arange(0, 20, 5) # [0, 5, 10, 15]
even_spread = np.linspace(0, 1, 5) # 5 evenly spaced numbers from 0 to 1
```

3.3 Indexing and slicing

```
scores = np.array([[80, 90], [60, 70], [55, 65]])

print(scores[0])      # first row:    [80 90]
print(scores[:, 0])   # first column: [80 60 55]
print(scores[1, 1])   # single value: 70
print(scores[scores > 65]) # every value greater than 65
```

3.4 Vectorised operations — no loops needed

```
marks = np.array([40, 60, 80, 100])

print(marks + 5)      # add 5 to every value:  [ 45 65  85 105]
print(marks * 2)      # double every value:    [ 80 120 160 200]
print(marks / 100)    # convert to fractions
print(marks > 60)     # a boolean array: [False False True True]

# Combine two arrays element by element
midterm = np.array([70, 65, 88, 90])
```

```
final = np.array([75, 80, 82, 95])
average = (midterm + final) / 2
print(average)
```

Why this matters

On a list of 4 numbers the speed difference is invisible. On a dataset with a million rows, a vectorised NumPy operation can be **50–100x faster** than a Python `for` loop, because NumPy runs the loop in fast, compiled C code behind the scenes instead of in slow Python bytecode.

3.5 Useful aggregate functions

```
marks = np.array([40, 60, 80, 100])

print(marks.mean())    # 70.0
print(marks.std())     # standard deviation
print(marks.sum())     # 280
print(marks.max(), marks.min())

# np.where: a vectorised "if-else"
result = np.where(marks >= 40, "Pass", "Fail")
print(result)          # ['Pass' 'Pass' 'Pass' 'Pass']
```

4 Pandas — DataFrames, groupby, merge, pivot

(75 minutes)

4.1 What is a DataFrame?

A **DataFrame** is pandas' name for a table: rows and named columns, just like a spreadsheet. Almost every dataset you load in this course — the IPL matches, the COVID-19 records, the HR analytics data — ends up as a DataFrame.

Key Idea

Think of a DataFrame as a dictionary of columns that all share the same row index. Each column can hold a different data type (numbers, text, dates), but every column has the same number of rows.

4.2 Creating and inspecting a DataFrame

```
import pandas as pd

df = pd.DataFrame({
    "Student": ["Aisha", "Ravi", "Meera", "Kabir"],
    "Marks":  [78, 55, 91, 40],
    "City":   ["Delhi", "Pune", "Delhi", "Lucknow"],
})

print(df.head())       # first 5 rows
print(df.shape)         # (4, 3) -- rows, columns
print(df.dtypes)        # data type of each column
print(df.describe())    # quick statistical summary of numeric columns
print(df.isna().sum()) # count of missing values per column
```

4.3 Selecting rows and columns

```
df["Marks"]                  # a single column (a pandas Series)
df[["Student", "Marks"]]     # more than one column
df[df["Marks"] >= 60]        # rows where the condition is True
df.loc[0, "Student"]         # row 0, column "Student", by label
df.iloc[0:2]                 # first two rows, by position
```

4.4 groupby — summarising data by category

This is the pandas operation you'll use the most. It follows the same "split – apply – combine" idea every time: split the data into groups, apply a calculation to each group, then combine the results back into a table.

Key Idea

`df.groupby("column").agg_function()` means: "for every unique value in *column*, calculate this separately." It's the pandas version of a pivot table's row grouping.

```
avg_marks_by_city = df.groupby("City")["Marks"].mean()
print(avg_marks_by_city)
```

```
# Multiple statistics at once
summary = df.groupby("City")["Marks"].agg(["mean", "max", "count"])
print(summary)
```

4.5 `merge` — combining two tables

`merge` is pandas' version of a SQL join: it lines up two DataFrames using a shared column (a "key") and combines their columns into one table.

```
cities = pd.DataFrame({
    "City": ["Delhi", "Pune", "Lucknow"],
    "State": ["Delhi", "Maharashtra", "Uttar Pradesh"],
})

merged = df.merge(cities, on="City", how="left")
print(merged)
```

Note

`how="left"` keeps every row from `df` even if there's no match in `cities`. Other common options are `"right"`, `"inner"` (only matching rows), and `"outer"` (everything from both sides).

4.6 `pivot_table` — reshaping data

A pivot table turns unique values from one column into new columns, which makes it easy to compare categories side by side.

```
pivot = df.pivot_table(
    values="Marks", index="City", aggfunc="mean"
)
print(pivot)

# A more typical example: rows = City, columns = Pass/Fail, values = count
df["Result"] = ["Pass" if m >= 40 else "Fail" for m in df["Marks"]]
pivot2 = df.pivot_table(
    values="Student", index="City", columns="Result", aggfunc="count", fill_value=0
)
print(pivot2)
```

Why this matters

`groupby`, `merge`, and `pivot_table` are the three tools you reach for whenever a manager asks "can you break this down by region / product / month?" Together they cover almost every real-world summarising question.

5 File I/O — CSV, JSON, Excel

(45 minutes)

Data rarely starts inside Python — it lives in files, databases, or on the web. This section covers the three formats you'll meet constantly: CSV, JSON, and Excel.

5.1 CSV — the universal format

```
import pandas as pd

# Reading a CSV -- works for a local file or a direct URL
df = pd.read_csv("students.csv")
df = pd.read_csv("https://example.com/students.csv")

# Writing a CSV
df.to_csv("students_cleaned.csv", index=False)
```

Note

Always pass `index=False` when saving, unless you specifically want pandas' row-number index saved as its own column. Forgetting this is the single most common CSV mistake.

5.2 JSON — nested, web-friendly data

```
import json

# Reading JSON directly into a DataFrame (works well for flat JSON)
df = pd.read_json("students.json")

# Reading JSON as a plain Python dictionary (better for nested structures)
with open("students.json") as f:
    data = json.load(f)

# Writing
df.to_json("students_out.json", orient="records", indent=2)
```

5.3 Excel — multi-sheet workbooks

```
# Reading a specific sheet
df = pd.read_excel("marks.xlsx", sheet_name="Sheet1")

# Reading every sheet at once (returns a dictionary of DataFrames)
all_sheets = pd.read_excel("marks.xlsx", sheet_name=None)

# Writing
df.to_excel("marks_cleaned.xlsx", index=False, sheet_name="Cleaned")
```

Note

Reading and writing Excel files needs the `openpyxl` package installed (`pip install openpyxl`). If you get an "Excel file format cannot be determined" error, this is almost always the missing piece.

5.4 Quick comparison

Format	Best for	Watch out for
CSV	Simple flat tables, fastest to read/write	No data types are stored — everything is re-guessed on load
JSON	Nested or web API data	Needs flattening before it looks like a table
Excel	Data shared with non-programmers, multiple sheets	Slower to read; needs `openpyxl`

6 Matplotlib & Seaborn for EDA

(60 minutes)

6.1 Why visualise at all?

A table of a thousand rows tells you almost nothing at a glance. A chart of the same rows can show a trend, an outlier, or a pattern in about two seconds. **Exploratory Data Analysis (EDA)** is the habit of looking at your data through charts before you model it.

Key Idea

Matplotlib is the foundational plotting library — flexible but a bit verbose. **Seaborn** is built on top of Matplotlib and gives you good-looking statistical charts (like boxplots and heatmaps) in far fewer lines. Most real projects use both together.

6.2 Matplotlib basics

```
import matplotlib.pyplot as plt

# Line chart
plt.plot(df["Marks"])
plt.title("Marks Trend")
plt.xlabel("Student index"); plt.ylabel("Marks")
plt.show()

# Bar chart
plt.bar(df["Student"], df["Marks"])
plt.show()

# Histogram -- shows the distribution/spread of one column
plt.hist(df["Marks"], bins=5)
plt.show()

# Scatter plot -- shows the relationship between two columns
plt.scatter(df["Marks"], df["Marks"] * 0.9)
plt.show()
```

6.3 Seaborn basics

```
import seaborn as sns

# Countplot -- how many rows fall into each category
sns.countplot(data=df, x="City")

# Boxplot -- spread, median, and outliers of a numeric column, by category
sns.boxplot(data=df, x="City", y="Marks")

# Heatmap -- great for correlation matrices
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
```

6.4 A few habits that make charts easier to read

- Always add a `title`, and label both axes.

- Rotate long category labels (`plt.xticks(rotation=45)`) so they don't overlap.
- Pick one colour palette and stay consistent across a report.
- Sort bar charts by value (largest to smallest) instead of leaving them alphabetical — it's much easier to compare.

Why this matters

The mini project that follows this handbook builds a full EDA report — more than ten charts — on India's real COVID-19 data, using exactly the Matplotlib and Seaborn patterns above.

What's Next

You now have the six building blocks of everyday data science work in Python. Open the **Module 1 Practice Notebook** and work through the same six sections using a real IPL match dataset — then move on to the **Mini Project**, where you'll use all of these tools together on a real, messy, government-sourced dataset: India's COVID-19 case and vaccination records.